Making Text Generation with LLMs **Faster**

Mike Erlihson, PhD, Head of AI at Stealth🤫

Outline:

Classic autoregressive LLM decoding, why is it slow?

Bottlenecks of autoregressive LLM decoding: memory or computations?

Blockwise **Parallel** Decoding and Classic Speculative Decoding

Adapting LLMs to Speculative Decoding(Medusa, Eagles…)

Parallel Fixed-Point Methods for LLM decoding

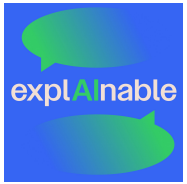
Long-context LLM parallel decoding: ring and tree attention

Training LLMs with multi-token prediction (e.g. DeepSeek)

Q & A(hopefully)

About Mike

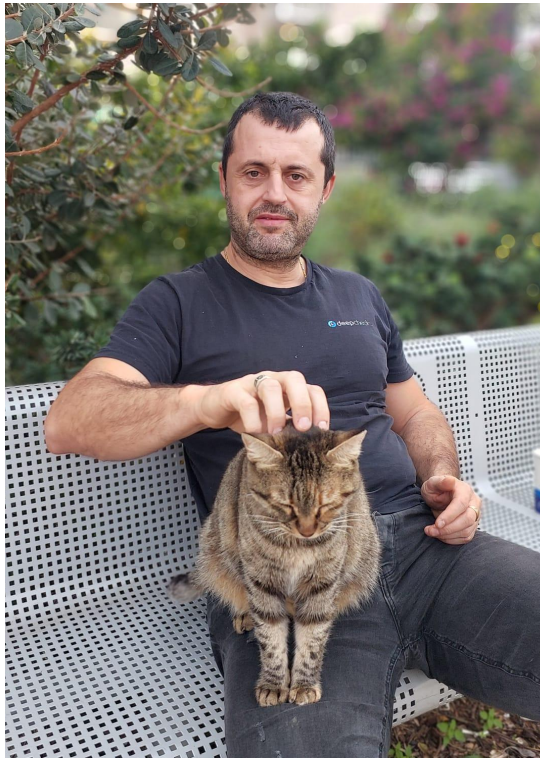
Math PhD



ExplAInable Podcast

Daily Paper Review

DS Decoded YouTube



~ 460 reviews of deep learning papers(Hebrew, English)
> 50 recorded podcasts about AI, ML, DS & math
> 60K Linkedin followers
> 2.5K subs on “Mathy AI” Telegram channels

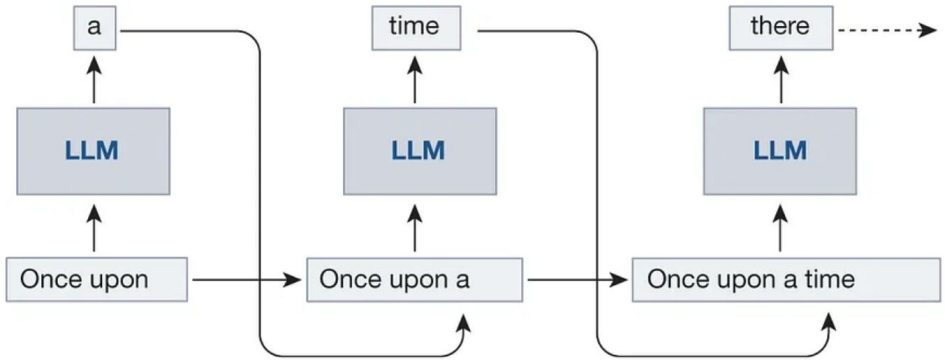
Twitter: https://x.com/MikeE_3_14
Telegram: https://t.me/MathyAIwithMike, https://t.me/science_and_ai_with_mike_english
Data Science Decoded: https://podcasters.spotify.com/pod/show/data-science-decoded

#deepnightlearners, #shorthebrewpapereviews, mike’s daily papers

How does an LLM generate text?

Token by token, based on the probability distribution over a fixed dictionary of tokens

Generating long texts token by token can be **very slow** - can we do better?

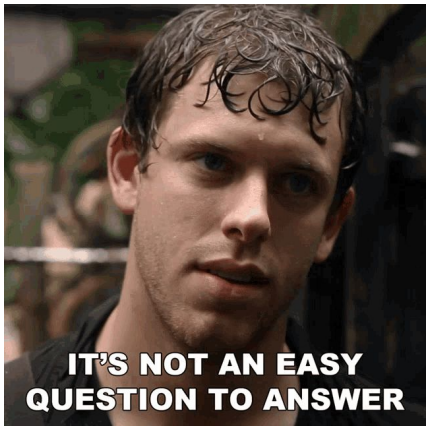


Parallel Decoding of LLMs - is it even possible?

Not a very trivial question actually 🤔

LLMs are trained to predict a next token given the preceding tokens (aka the context)

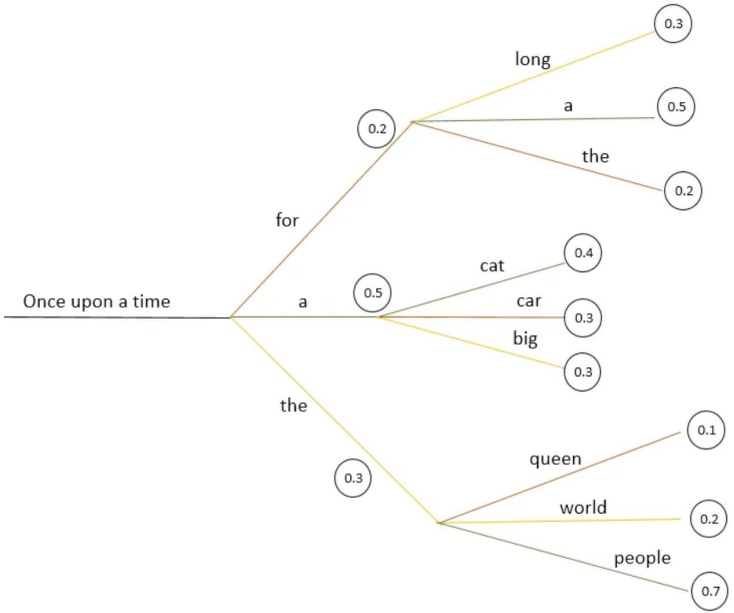
Can we generate text in a different (≠ training) fashion?



But what about beam 💣💣 search - is it parallel decoding?

Maintains a beam of the K most probable sequences for each token

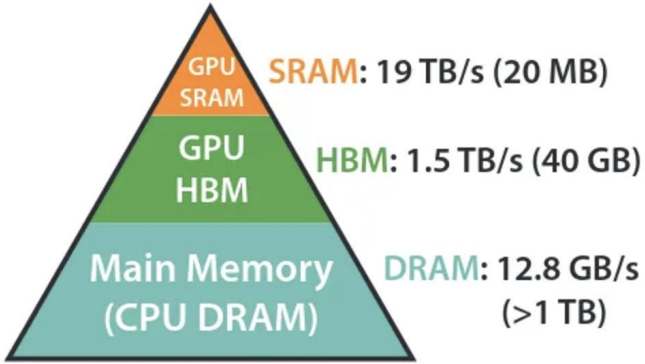
But it is still done token-by-token - **NOT PARALLEL**



Can we accelerate this token-by-token decoding?

The answer is actually yes 🤗, but to understand it we need to dive into….

🤯 the GPU memory hierarchy… 🤯



Memory Hierarchy with Bandwidth & Memory Size

Small but fast SRAM

Big but slow HBM (high-bandwidth memory)

Huge but very slow CPU DRAM

Leverage GPU Memory Hierarchy 4Faster LLM Decoding

LLMs Inference is often **not bottlenecked** on arithmetic ops, but on memory bandwidth and communication

The main **latency bottleneck** of a decoding step is caused by the transfer of all LLM params from High-Bandwidth Memory(HBM) to the GPU on-chip cache(SRAM)

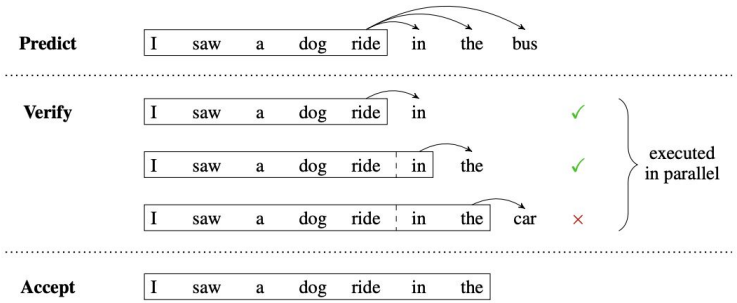
🔑 Key Observation:
🚀 More Parallel Computation
📉 Less Memory Transfer (to GPU from Memory)

Early Attempts of Parallel LLM Decoding

Blockwise Parallel Decoding for Deep Autoregressive Models (2018)

k LLMs: j-th LLM is trained to predict probs for the (i + j)-th token, given i first tokens (1st model is a regular LLM)

Produces the same prediction as the standard decoding but uses k times less steps.



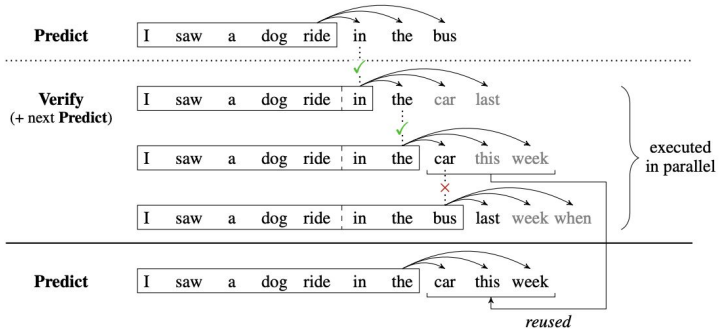


Figure 2: Combining the scoring and proposal models allows us to merge the previous verification substep with the next prediction substep. This makes it feasible to call the model just once per iteration rather than twice, halving the number of model invocations required for decoding.

Blockwise Parallel Decoding for Deep Autoregressive Models

But why is it good? 🤔💭

🧮 Computations are fast 🚀 and we can compute all k models' predictions quite efficiently

🥺 But we still a lot of SRAM(fast memory) in GPU to predict simultaneously with k (!!) models

Can we do better? May be with a just single additional model? 🤔💭

The answer is apparently ✅ ***Yes***

Speculative Decoding: Small LM with Large LM

🤔💭 We can leverage a **single** smaller & faster model instead our huge LLM?

🚫 Ok, but then the performance may degrade, right?

🤩 Not necessarily, if we smartly combine it with our powerful LLM

🎰 How? With speculative decoding, introduced in Nov, 2022 (Y. Leviathan et al**)

** Based on Speculative Decoding: Exploiting Speculative Execution for Accelerating Seq2seq Generation, a bit simpler approach

Fast Inference from Transformers via Speculative Decoding

For the context tokens: x_0,..., x_t:

- Use the faster(weaker) model M_q to generate γ next tokens x_{t+i}, i = 1,.., γ

- Evaluate p(x_{t+i} | x_j, j=0,..., t+i-1} **in parallel**

- Accept all tokens that can lead to an **identical to Mp distribution**

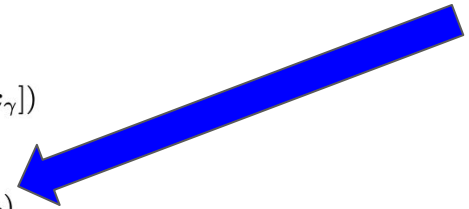
- Sample an additional token from an adjusted distribution to fix the 1st rejected one, or to add an additional one if they are all accepted.

**** A similar method was proposed in a paper by Google 3 months later**

Fast Inference from Transformers via Speculative Decoding

Algorithm 1 SpeculativeDecodingStep

Inputs: $M_p, M_q, prefix$.
▷ Sample γ guesses $x_{1,\ldots,\gamma}$ from M_q autoregressively.
for $i = 1$ **to** γ **do**
  $q_i(x) \leftarrow M_q(prefix + [x_1, \ldots, x_{i-1}])$
  $x_i \sim q_i(x)$
end for
▷ Run M_p in parallel.
$p_1(x), \ldots, p_{\gamma+1}(x) \leftarrow$
  $M_p(prefix), \ldots, M_p(prefix + [x_1, \ldots, x_\gamma])$
▷ Determine the number of accepted guesses n.
$r_1 \sim U(0,1), \ldots, r_\gamma \sim U(0,1)$
$n \leftarrow \min(\{i - 1 \mid 1 \leq i \leq \gamma, r_i > \frac{p_i(x)}{q_i(x)}\} \cup \{\gamma\})$
▷ Adjust the distribution from M_p if needed.
$p'(x) \leftarrow p_{n+1}(x)$
if $n < \gamma$ **then**
  $p'(x) \leftarrow norm(max(0, p_{n+1}(x) - q_{n+1}(x)))$
end if
▷ Return one token from M_p, and n tokens from M_q.
$t \sim p'(x)$
return $prefix + [x_1, \ldots, x_n, t]$



“Comparing” Mp and Mq distributions
(~rejection sampling)

Speculative vs. AR Decoding: General Comparison

Algorithm 1 Autoregressive Decoding

Require: Language model $\mathcal{M}_q$, input sequence $x_1, \ldots, x_t$, and target sequence length T;

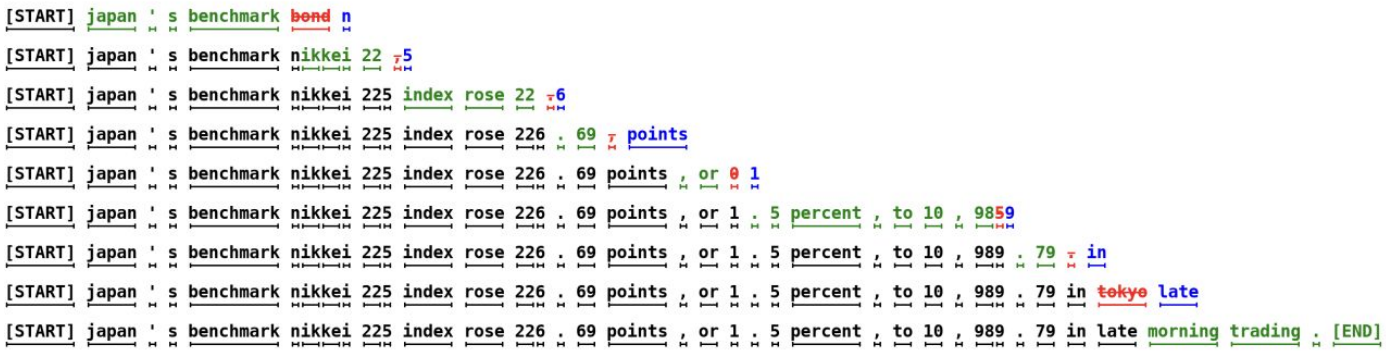
initialize $n \leftarrow t$
while $n < T$ **do**
    Set $q_{n+1} \leftarrow \mathcal{M}_q\left(x \mid x_{<n+1}\right)$
    Sample $x_{n+1} \sim q_{n+1}$
    $n \leftarrow n+1$
end while

Algorithm 2 Speculative Decoding

Require: Target language model $\mathcal{M}_q$, draft model $\mathcal{M}_p$, input sequence $x_1, \ldots, x_t$, block size K, target sequence length T, drafting strategy DRAFT, verification criterion VERIFY, and correction strategy CORRECT;

initialize $n \leftarrow t$
while $n < T$ **do**
    // Drafting: obtain distributions from $\mathcal{M}_p$ efficiently
    Set $p_1, \ldots, p_K \leftarrow \text{DRAFT}\left(x_{\leq n}, \mathcal{M}_p\right)$
    // Drafting: sample K drafted tokens
    Sample $\widetilde{x}_i \sim p_i, i = 1, \ldots, K$
    // Verification: compute $K+1$ distributions in parallel
    Set $q_i \leftarrow \mathcal{M}_q\left(x \mid x_{\leq n}, \widetilde{x}_{<i}\right), i = 1, \ldots, K+1$
    // Verification: verify each drafted token
    for $i = 1 : K$ **do**
        if $\text{VERIFY}\left(\widetilde{x}_i, p_i, q_i\right)$ **then**
            Set $x_{n+i} \leftarrow \widetilde{x}_i$ and $n \leftarrow n+1$
        else
            $x_{n+i} \leftarrow \text{CORRECT}\left(p_i, q_i\right)$
            and Exit for loop.
        end if
    end for
    If all drafted tokens are accepted, sample next token $x_{n+1} \sim q_{K+1}$ and set $n \leftarrow n+1$.
end while

Fast Inference from Transformers via Speculative Decoding



🟢 tokens are the tokens made by the fast model that the target model accepted

🔴 and 🔵 tokens are the rejected tokens and their corrections

In the 1st line the target model was run only once, and 5 tokens were generated.

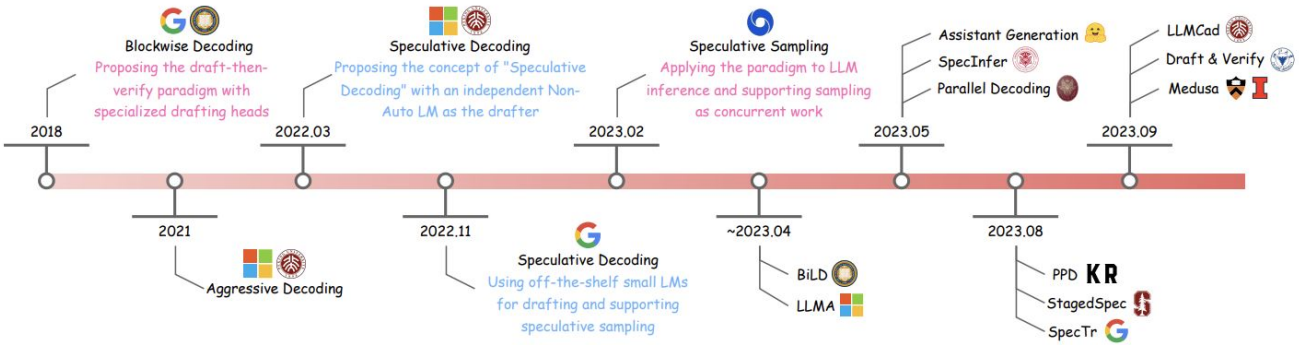
Fast Inference from Transformers via Speculative Decoding

Optimization questions:

How many tokens should we generate with the approximation model?

How to choose the approximation(draft) LLM?

Maybe we should fine-tune the approximation model for more efficient performance in the speculative decoding



💬 Let’s discuss a couple(dozen 😁) of them 🗣️

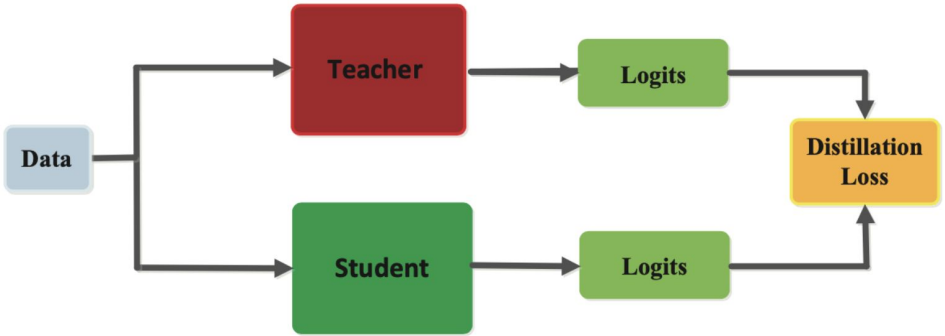
DistillSpec: Improving SD via Knowledge Distillation

Knowledge distillation to better align the weak model with the target model before applying SD

The distillation data was generated by the weak model (kinda on-policy)

Total variation distance between token distributions (instead of KL)

10 − 45% speedups over standard SD on a range of benchmarks

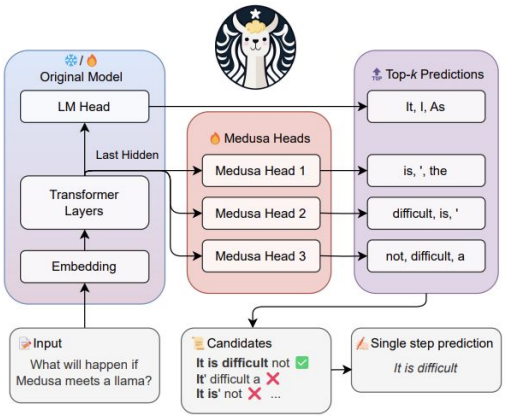


MEDUSA: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads

K prediction heads added to the target model(instead of its head) - **no draft model**

k-th head is trained to predict (t + k + 1)-th token (the original LLM head predicts the (t + 1)-th token)

Each head predicts s_k tokens with the highest probabilities



$$p_t^{(k)} = \text{softmax}\left(W_2^{(k)} \cdot \left(\text{SiLU}(W_1^{(k)} \cdot h_t) + h_t\right)\right),$$
$$\text{where } W_2^{(k)} \in \mathbb{R}^{d \times V}, W_1^{(k)} \in \mathbb{R}^{d \times d}.$$

MEDUSA, cont

Builds a tree structure where s_k(= predicted tokens) exist at the k-th level

Construct the most “plausible” K tokens sequence from this tree, giving “preference” to high prob. tokens(instead of rejection sampling)

The final prediction is determined by the longest accepted prefix among all candidates

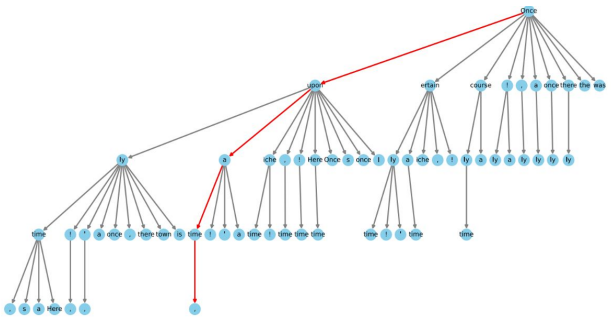


Figure 6. Visualization of a sparse tree setting for MEDUSA-2 Vicuna-7B. The tree has 64 nodes representing candidate tokens and a depth of 4 which indicates 4 MEDUSA heads involved in calculation. Each node indicates a token from a top-k prediction of a MEDUSA head, and the edges show the connections between them. The red lines highlight the path that correctly predicts the future tokens.

EAGLE: Greater Language-model Efficiency

Processing(predicting) at the model feature level is “simpler” than at the token level.

Feature sequences exhibit more regularity than token sequences

Main Idea:

- Autoregressively predict at the feature level
- Derive tokens using the original LLM head
- Better than directly predicting tokens.

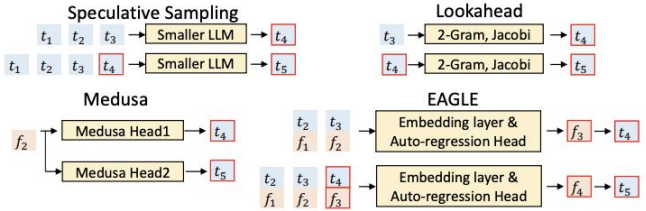


Figure 5: A comparison of the methods for drafting the fourth and fifth tokens, t_4 and t_5. t (represented by blue blocks) denotes tokens, and f (orange blocks) signifies the features, with subscripts indicating their positions in the sequence. The red border indicates the predictions of the draft model. For simplicity, the n in the n-gram for Lookahead, as shown in the figure, has been set to 2.

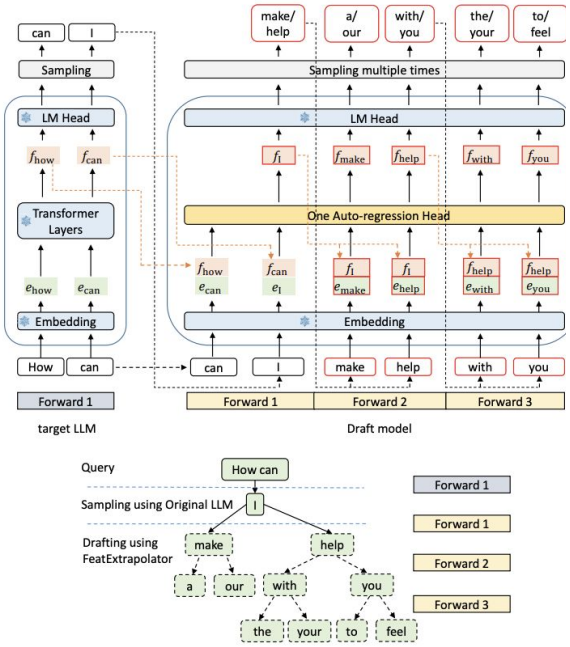
EAGLE: Greater Language-model Efficiency

An autoregressive trainable head added before the original LLM prediction head

This head is trained to predict i-th feature and (i+1)-th token from the preceding features & tokens

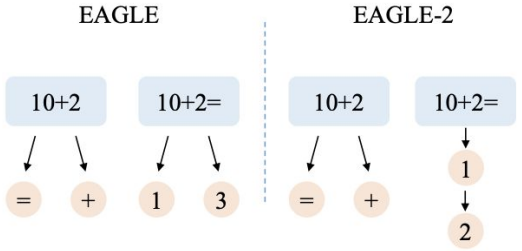
A prediction tree is built similarly to Medusa

Draft model := Original LLM + Trainable Head



EAGLE2:

Dynamically adjustable draft tree (use high probable tokens, not top-k) to build the tree (leaves with different depths)



Reranking phase: Select token sequences with higher likelihoods(possibly different lengths) as input for the original LLM during the verification phase

Improves acceptance rate of the draft tokens

Speculative Decoding: Draft Model Choice Strategies 📝

Smaller variants of the same model family: smaller model from the same architecture family

- Token distributions tend to be more aligned, increasing acceptance rates

Distilled models: Mimic the larger model's outputs while being faster

- Directly optimized to match the target model's token distribution through knowledge distillation
- Distilled models can maintain 80-90% acceptance rates while being 3-5x faster

👉👉👉👉

Speculative Decoding: Draft Model Choice Strategies 📝

Quantized versions of the target model: same weights but with less bits

- Effective for on-device applications or memory-constrained envs
- Advanced q-techniques have significantly reduced quality degradation(quant. aware training

Self-drafting approaches: Use early layers/subset of the target model

- Avoids alignment issues between different models
- Leverages early exiting techniques(shallow layers make predictions when confidence is high)
- Speculative beam search techniques can generate & evaluate multiple candidates simultaneously

Now Fasten Your Seat Belts!

Be Prepared for Some Tough Math

Fast LLM Decoding with NonLinear Systems of Equations

2.2.1. Nonlinear Jacobi Iteration

To solve a system of equations like Eq. (2), iterative methods start from an initial guess $\mathbf{x}^0 \triangleq (x_1^0, x_2^0, \cdots, x_N^0)$ of the solution, and gradually improve it through fixed-point iterations. We let $\mathbf{x}^k = (x_1^k, x_2^k, \cdots, x_N^k)$ denote the solution obtained at the k-th iteration. Given $\mathbf{x}^k$, the nonlinear Jacobi iteration produces $\mathbf{x}^{k+1}$ by solving each univariate equation for $i = 1, 2, \cdots, N$:

$$f_i(x_1^k, \cdots, x_{i-1}^k, x_i, x_{i+1}^k, \cdots, x_N^k) = 0 \qquad (3)$$

for x_i. We then set $x_i^{k+1} = x_i$ for all i. The process stops when it reaches a fixed point, or $\mathbf{x}^{k+1}$ is sufficiently similar to $\mathbf{x}^k$ as measured by the *forward difference* $\left\|\mathbf{x}^{k+1} - \mathbf{x}^k\right\| \leq \epsilon$, where $\epsilon > 0$ is a tolerance threshold. Crucially, all the N univariate equations involved can be solved *in parallel* since there is no dependency among them.

2.2.2. Nonlinear Gauss-Seidel (GS) Iteration

Nonlinear Gauss-Seidel (GS) iteration is another iterative solver for systems of nonlinear equations. Similar to Eq. (3), the k-th step of nonlinear GS is to solve

$$f_i(x_1^{k+1}, \cdots, x_{i-1}^{k+1}, x_i, x_{i+1}^k, \cdots, x_N^k) = 0 \qquad (4)$$

for x_i and to set $x_i^{k+1} = x_i$ for $i = 1, 2, \cdots, N$. The process stops when it reaches a fixed point, or $\left\|\mathbf{x}^{k+1} - \mathbf{x}^k\right\| \leq \epsilon$. Different from Eq. (3), GS updates leverage the new solutions as soon as they are available. This creates data dependency among adjacent univariate equations and therefore requires N sequential computations to get $\mathbf{x}^{k+1}$ from $\mathbf{x}^k$. Assuming that each univariate equation of Eq. (3) and Eq. (4) takes the same time to solve, one GS iteration costs as much time as N parallel Jacobi iterations.

Decoding LLMs with Gauss-Seidel and Jacobi Methods

How is this math stuff connected to LLMs?

Decoding LLMs with Jacobi Fixed-Point Iterations

For a prompt x and an LLM p(*|**x**), standard AR generates tokens with

$$y_i = \arg\max_y p(y \mid y_{:i}, \mathbf{x}) \quad \text{for } i = 1, \ldots, n$$

Jacobi/GZ decoding parallelizes the LLM inference turning it into solving a SNE.

$\boldsymbol{x}$: prompt, $\boldsymbol{y} = [y_1, y_2, \ldots, y_m]$: m tokens to decode, $p(\boldsymbol{y}|\boldsymbol{x})$: LLM distribution

Define: $f(y_i, \boldsymbol{y}_{1:i-1}, \boldsymbol{x}) = y_i - \operatorname{argmax} p(y_i|\boldsymbol{y}_{1:i-1}, \boldsymbol{x})$

$$\begin{cases} y_1 = \operatorname{argmax} p(y_1|\boldsymbol{x}) \\ y_2 = \operatorname{argmax} p(y_2|y_1, \boldsymbol{x}) \\ \vdots \\ y_m = \operatorname{argmax} p(y_m|\boldsymbol{y}_{1:m-1}, \boldsymbol{x}) \end{cases} \quad \Leftrightarrow \quad \begin{cases} f(y_1, \boldsymbol{x}) = 0 \\ f(y_2, y_1, \boldsymbol{x}) = 0 \\ \vdots \\ f(y_m, \boldsymbol{y}_{1:m-1}, \boldsymbol{x}) = 0 \end{cases}$$

Autoregressive decoding

Non-linear system with
m variables and m equations

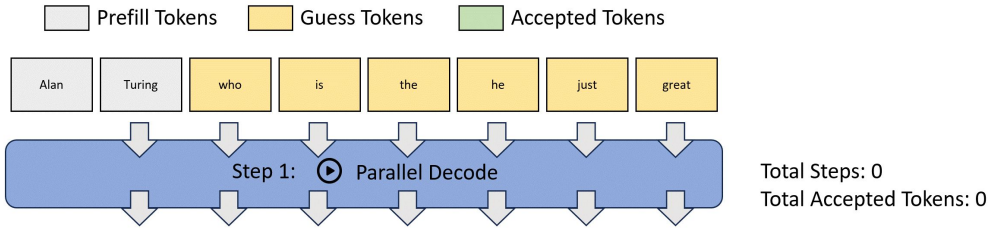
Solve these NSE (predict tokens) in **PARALLEL!**

Start from initial guess for all variables **y =** [y_1, y_2, …, y_m]

Compute new **y'** for each equation with the previous **y**

Update **y** to the newly computed **y'**

Repeat until **y** is sufficiently close to **y'** (or m iterations for m tokens)

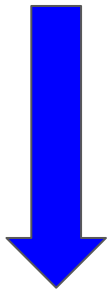


Decoding LLMs with Jacobi Fixed-Point Iterations

This process is guaranteed to converge at max m iterations (for m predicted tokens)

Why? ⇒ i-th token is always predicted correct for i-th Jacobi iteration

Each Jacobi iteration step might predict more than one correct token



Potentially accelerates AR LLM decoding


PROBLEM SOLVED


UHMM...
NOT REALLY
SNL

Limitations of Vanilla Jacobi Method for LLMs

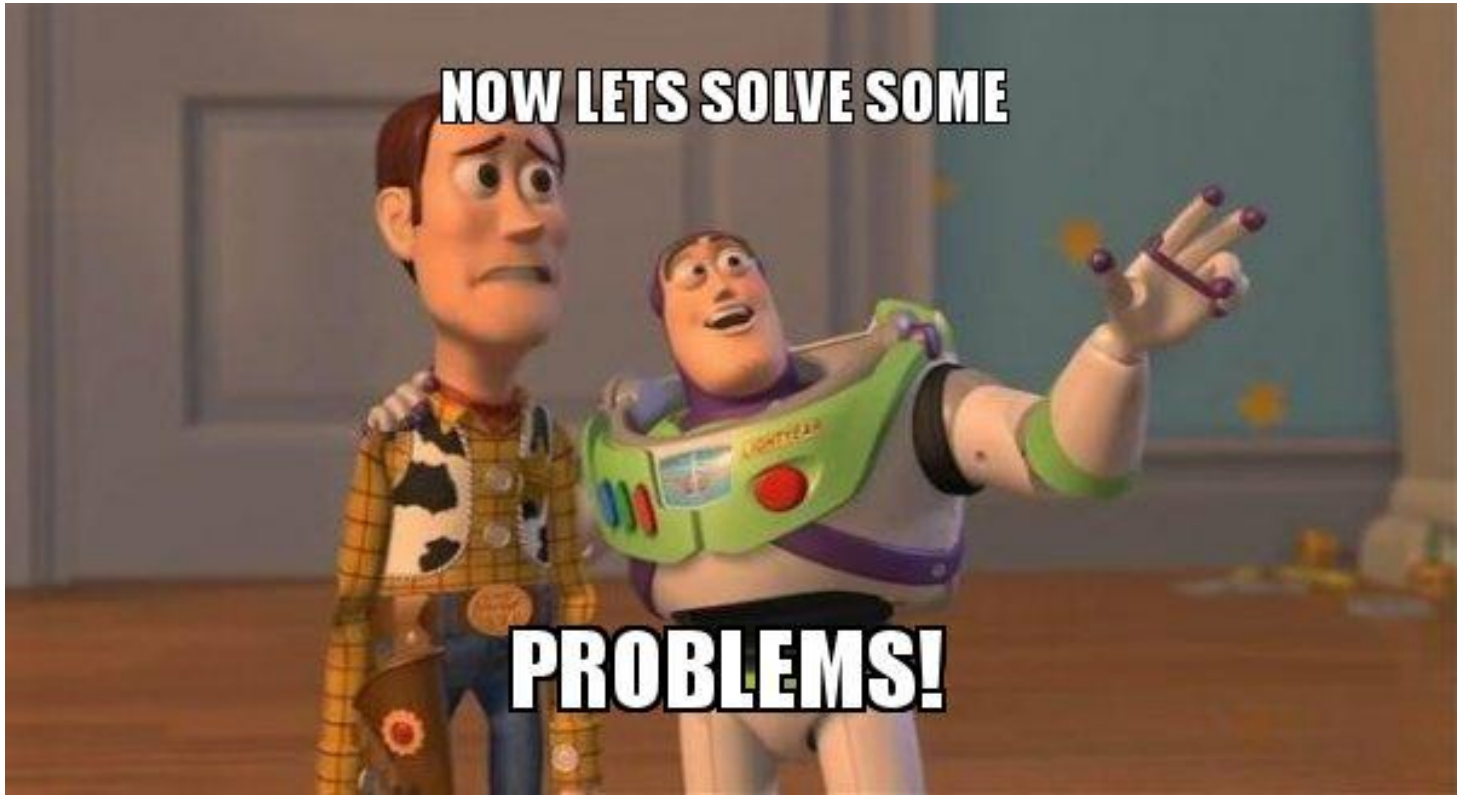
Vanilla Jacobi decoding for LLMs shows only **~1.05 speedup** over AR decoding

An LLM rarely yields a correct token when there are errors in its preceding tokens.

Most iterations gain only 1 correction for a sequence, resulting in a longer trajectory

Even when tokens are correctly predicted, they are often replaced in later iterations

Few iterations successfully achieve the **real simultaneous decoding**


NOW LETS SOLVE SOME
PROBLEMS!

Lookahead Decoding +Jacobi: Generate parallel n-grams

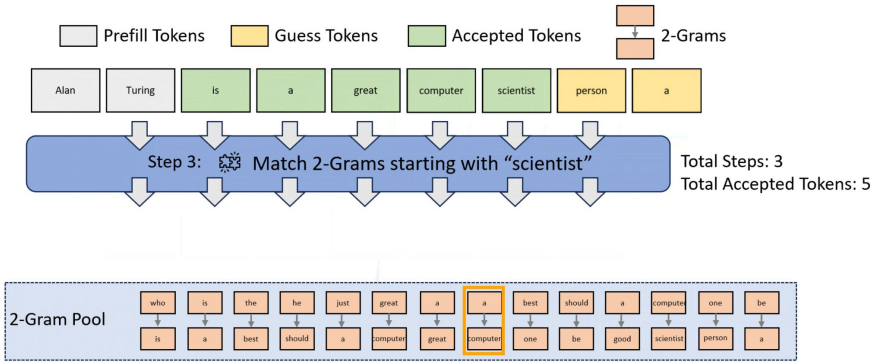
A new token is decoded based on its historical values from previous Jacobi iterations

Build trajectory of historical tokens at each token position => N-grams

N iterations => a N-gram(N tokens forward) formed at each token position

Collect and cache these N-grams from their trajectories.

Utilize cached N-grams for the next iteration, starting from the last accepted token



Lookeahead Decoding with Jacobi: Main Principles

Do a Jacobi iteration for several tokens, with the trajectory formed by the previous N-1 iteration(+ newly generated tokens)

Append N-grams to the current input (with the same start token) and verify

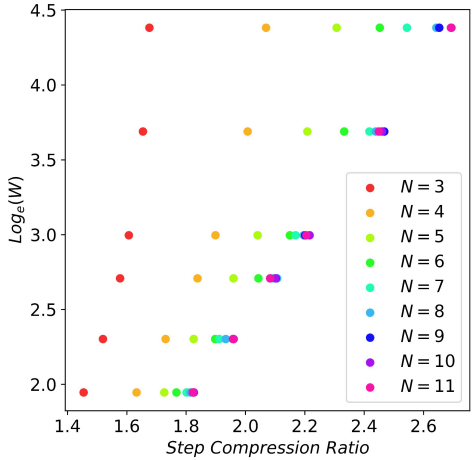
More chance to find sequences of tokens that will be accepted (higher acceptance rate)

As the n-gram cache grows => more n-grams likely start with the same token => higher verification cost.

Lookeahead Decoding with Jacobi: Some Visuals

[Lookahead decoding: visually]

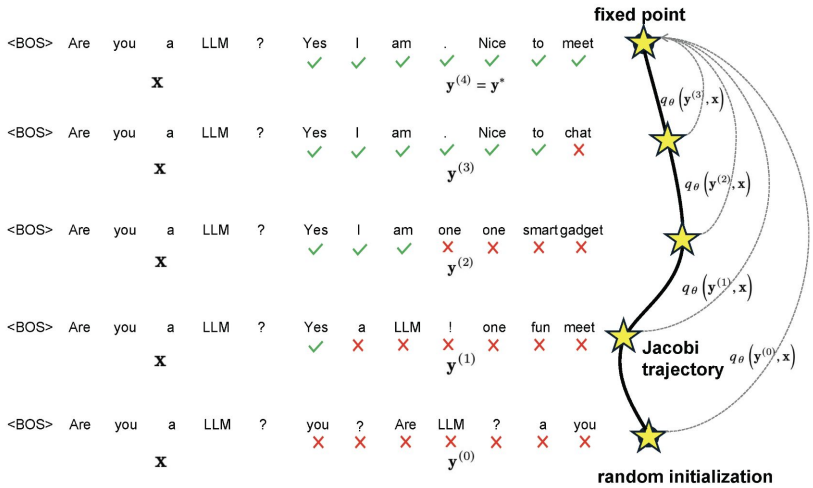
More Iterations => Higher Token Acceptance Rate => Bigger Cache Required



Consistency LLMs: Train a model to adapt to Jacobi

Adapt pre-trained LLMs so that they can consistently map any point (= token sequence) on the J-trajectory to the fixed point(correct tokens)

Use sampled J-trajectories to train the model to encourage single-step convergence of J-iterations.



Consistency LLMs: Training Procedure

Perform Jacobi decoding for n-token example sub-sequences & concatenate them (=> full sequence).

Each generated subsequence serves as a training sample

Training: combination of 2 losses

Consistency: encourages prediction of multiple tokens simultaneously, promoting efficient parallel decoding

Autoregressive(AR): maintains the generation quality by preventing the CLLM from deviating from the target LLM.

C-LLMs: Losses

$$\mathcal{L}_{\text{GC}} = \mathop{\mathbb{E}}_{(\mathbf{x},\mathcal{J})\sim\mathcal{D},\mathbf{y}\sim\mathcal{J}}\Big[\sum_{i=1}^{n} D(q_{\theta^-}(\cdot|\mathbf{y}^*_{:i},\mathbf{x}))||q_\theta(\cdot|\mathbf{y}_{:i},\mathbf{x})\Big]$$

Global consistency

$$\mathcal{L}_{\text{LC}} = \mathop{\mathbb{E}}_{(\mathbf{x},\mathcal{J})\sim\mathcal{D},(\mathbf{y}^{(j)},\mathbf{y}^{(j+1)})\sim\mathcal{J}}\Big[\sum_{i=1}^{n} D(q_{\theta^-}(\cdot|\mathbf{y}^{(j+1)}_{:i},\mathbf{x}))||q_\theta(\cdot|\mathbf{y}^{(j)}_{:i},\mathbf{x})\Big]$$

Local consistency

$$\mathcal{L}_{\text{AR}} = \mathop{\mathbb{E}}_{(\mathbf{x},\mathbf{l})\sim\mathcal{D}}\Big[-\sum_{i=1}^{N}\log q_\theta(l_i|\mathbf{l}_{:i},\mathbf{x})\Big]$$

Standard autoregressive loss

Similarity to Consistency Diffusion Models

Fixed-Point Learning: Leverage **consistency loss**, ensuring that their predictions remain stable and do not drift when iteratively refined.

CDMs: Denoising steps across different noise levels lead to the same final clean sample.

CLLMs: Allows multiple tokens to be predicted in parallel w/o breaking sequence coherence.

Fast Inference: Reduces the number of iterative steps needed for high-quality output generation.

CDM: Accelerate image generation by skipping multiple diffusion steps while maintaining high fidelity.

CLLMs: Parallel decoding, predicting multiple tokens at once instead of sequential token-by-token decoding.

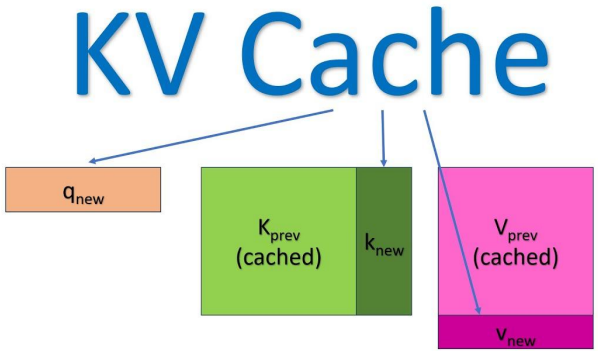

THE TIME HAS COME!
TO TALK OF OTHER THINGS.
imgflip.com

Faster Decoding with Multiple GPUs for Long Contexts

Remember KV-Cache?

For long texts the KV cache becomes the main memory bottleneck, forcing to limit the maximal sequence length(context window)

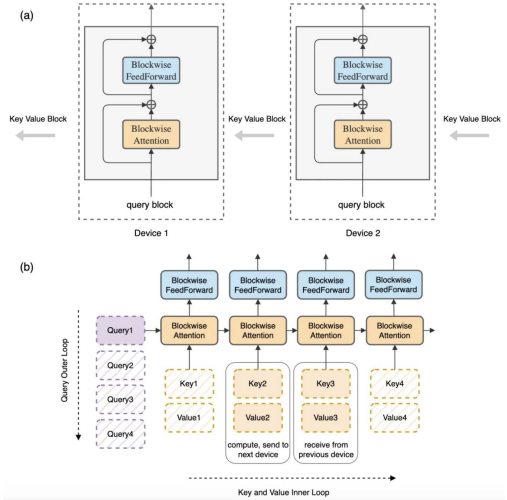
Can we accelerate the decoding process and ease RAM requirements?



Ring Attention: Parallel Attention Computation

Break them into ‘blocks’ and send each block to a different GPU.

If we have a very long sequence of 100K tokens and 10 GPUs, each GPU receives on average 10K tokens.



Example: “The quick brown fox jumps over the lazy dog”

“The quick brown” => GPU1, “fox jumps over” => GPU2, “the lazy dog” => GPU3

Outer loop. Q-vectors of [“The”, “quick” and “brown”] go to GPU1….

Inner loop. Each GPU computes self-attention(SA) over its Qs, and its FFNs.

But:

Qs of a token needs to be multiplied with the K vectors of every preceding token.

“Fox” is assigned to GPU2, it must see the Ks & Vs of [“The”, “quick”, & “fox”] of GPU1

Ring Attention: Parallel Attention Computation, Cont

GPUs must share the Ks & Vs with each other => here comes the **Ring** 💍🛟

In parallel to computing the inner loop SA and FFN, each GPU sends its Ks and Vs to the next GPU while receiving that info from the previous GPU.

Why and when does it work?

As long as GPU computation takes longer than transferring Ks and Vs between GPUs, communication causes no overhead: the GPU remains busy with its calculations

Tree Attention: Better than the Ring 💍🛟

Formulates a **scalar energy function** whose gradient directly **computes the SA**.

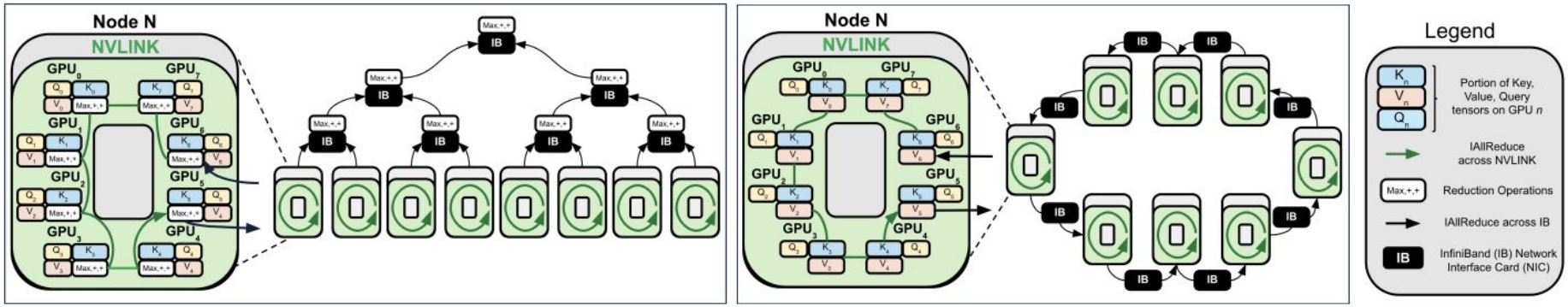
Recasts SA as **an energy minimization problem**: parallels to energy-based models as Hopfield Networks (converges to stable fixed points iteratively)

Restructures SA computation into a hierarchical tree topology; reduces the number of communication steps from O(N) to O(logN) when distributed across multiple GPUs.

However, while it has long been known that the softmax operation can be derived as the gradient of the following scalar function:

$$\partial_{z_j} \log \sum_{a=1}^{n} \exp(z_a) = \frac{e^{z_j}}{\sum_{a=1}^{n} e^{z_a}} = \text{softmax}(z_j), \qquad (1)$$

Tree Attention vs. Ring Attention: Visuals



(a)Multi Node **Tree Attention** (Ours)

(b)Multi Node **Ring Attention**

FIG. 1: Ring and **Tree Attention** Topologies. Due to the associative properties of the logsumexp and max operations of **Tree Attention** (Fig. 1(a)), is possible to structure the reduction across the sequence as a tree, requiring asymptotically fewer communication steps than **Ring Attention** (Fig. 1(b)) as well as less memory and communications volume.

Tree Attention: More Explanations

Instead of each GPU communicating with all other GPUs, each GPU interacts with a limited subset of GPUs at each stage, following a tree-like topology.

Significantly optimizes message passing, reduces unnecessary data transfers & bottlenecks.

Achieves faster decoding without sacrificing accuracy:

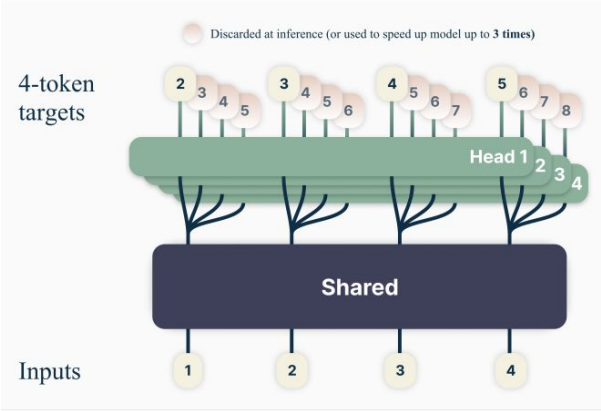
- Less waiting time between GPUs
- Faster overall computation
- More efficient scaling to longer context

Training a LLM to predict multiple tokens (Meta, DeepSeek)

Predicts multiple tokens simultaneously, improving training speed & GPU utilization

Enhances the ability to capture & generate coherent text, benefiting larger models & complex tasks.

Speeds up decoding from the NTP head with variety of self-speculative decoding methods



What did we learn today?

- Why the AR LLM decoding is slow?
- What are the main decoding bottleneck (data transfers)?
- A variety of the methods for fast LLM decoding from 2 families
  - Speculative decoding
  - Jacobi fixed-point iterations
- Fast Decoding if you have multiple GPUs
- Multi-token prediction can make decoding faster


YOUR
QUESTION
TIME